

Algorithmen & Datenstrukturen

Woche 5

Marius Tomek, Nicolas Wehrli, Tim Rieder

25. Oktober 2022

ETH Zürich

Kurze Kommentare zur letzten Serie

Sortieren

Heapsort

Mergesort

Quicksort

Homework 04

Kahoot

Peergrading 4.4

Kurze Kommentare zur letzten Serie

Sortieren

Wir betrachten einen Max-Heap von Grösse $n \in \mathbb{N}$:

Heap-Condition: Alle Knoten sind grösser oder gleich wie ihre Nachfolger.

In einem Array A :

Für ein $k \in \mathbb{N}, k < n$:

$$((2k < n) \implies (A[k] \geq A[2k]))$$

und

$$((2k + 1 < n) \implies (A[k] \geq A[2k + 1]))$$

Algorithm 1 Heapsort(A)

1: **for** $i \leftarrow \lfloor N/2 \rfloor$ downto 1 **do**

2: Heapify(A, i, n)

▷ We build the Heap

3: **for** $m \leftarrow n$ downto 2 **do**

4: Swap $A[m]$ and $A[1]$

▷ Extract the Maximum

5: Heapify($A, 1, m - 1$)

▷ Restore the Heap Condition

Heapsort – Heapify

Algorithm 2 Heapify(A, i, m)

1: while $2 \cdot i \leq m$ do	▷ while $A[i]$ has successors
2: $j \leftarrow 2 \cdot i$	▷ Set j to the index of the left successor
3: if $j + 1 \leq m$ then	▷ Check if there's also a right successor
4: if $A[j] < A[j + 1]$ then $j \leftarrow j + 1$	▷ Choose the bigger one
5: if $A[i] \geq A[j]$ then STOP	▷ Check Heap-Condition
6: Swap $A[i]$ with $A[j]$	
7: $i \leftarrow j$	▷ Continue after swap

Algorithm 3 Mergesort(A, l, r)

```
1: if  $l < r$  then  
2:    $m \leftarrow \lfloor (l + r) / 2 \rfloor$                                 ▷ Find middle  
3:   Mergesort( $A, l, m$ )                                         ▷ left half recursively  
4:   Mergesort( $A, m + 1, r$ )                                     ▷ right half recursively  
5:   Merge( $A, l, m, r$ )                                         ▷ Merge the two halves
```

Mergesort – Merge

Algorithm 4 Merge(A, l, m, r)

```
1:  $B \leftarrow \text{new Array}[r - l + 1]$ 
2:  $i \leftarrow l; j \leftarrow m + 1; k \leftarrow 1$ 
3: while  $i \leq m$  and  $j \leq r$  do                                ▷ Repeat until one half is inserted
4:   if  $A[i] \leq A[j]$  then  $B[k] \leftarrow A[i]; i \leftarrow i + 1$ 
5:   else  $B[k] \leftarrow A[j]; j \leftarrow j + 1$ 
6:    $k \leftarrow k + 1$ 
7: while  $i \leq m$  do                                              ▷ Attach the rest of the other half
8:    $B[k] \leftarrow A[i]; i \leftarrow i + 1; k \leftarrow k + 1$ 
9: while  $j \leq r$  do
10:   $B[k] \leftarrow A[j]; j \leftarrow j + 1; k \leftarrow k + 1$ 
11: for  $h \leftarrow l$  to  $r$  do                                       ▷ Copy back from  $B$ 
12:   $A[h] \leftarrow B[h - l + 1]$ 
```

Algorithm 5 Quicksort(A, l, r)

```
1: if  $l < r$  then  
2:    $k \leftarrow \text{Partition}(A, l, r)$   
3:   Quicksort( $A, l, k - 1$ )  
4:   Quicksort( $A, k, r$ )
```

Algorithm 6 Partition(A, l, r)

```
1:  $i \leftarrow l$ 
2:  $j \leftarrow r - 1$ 
3:  $p \leftarrow A[r]$ 
4: while  $i < j$  do
5:   while  $i < r$  and  $A[i] < p$  do  $i \leftarrow i + 1$ 
6:   while  $j > l$  and  $A[j] > p$  do  $j \leftarrow j - 1$ 
7:   if  $i < j$  then Swap  $A[i]$  and  $A[j]$ 
8: Swap  $A[i]$  and  $A[r]$ 
9: return  $i$ 
```

Homework 04

Kahoot

Kahoot Woche 06

Peergrading 4.4
